AIAC

ASSIGNMENT 10.3

NAME: T. Prashanthi

2303A52495

BATCH: 50

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):
result = 1
for i in range(1, n):
result = result * i
return result
```

Instructions:

1. Run the code and test it with factorial(5).

2. Use an AI assistant to:

o Identify the logical bug in the code.

o Explain why the bug occurs (e.g., off-by-one error).

o Provide a corrected version.

3. Compare the AI's corrected code with your own manual fix.

4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Expected Output:

Corrected function should return 120 for factorial(5).

PROMPT: # Analyze and fix this factorial function: Identify the logical bug,Explain the reason for the bug,

Provide the corrected code,Ensure factorial(5) returns 120,Handle edge cases like 0 and negative numbers

```
def factorial(n):
    result = 1
    for i in range(1, n):
        result = result * i
    return result
```

CODE:

```
Lab 10-AIAC.py > factorial
# Analyze and fix this factorial function: Identify the logical bug,Explain the reason for the bug,
# Provide the corrected code,Ensure factorial(5) returns 120,Handle edge cases like 0 and negative numbers


def factorial(n):
    result = 1
    for i in range(1, n):
        result = result * i
    return result
# Logical Bug: The loop in the factorial function runs from 1 to n-1, which means it does not include n in the multiplication. Th
# Reason for the Bug: The factorial of a number n is defined as the product of all positive integers from 1 to n. By excluding n
# Corrected Code:
def factorial(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers.")
    elif n == 0:
        return 1
    else:
        result = 1
        for i in range(1, n + 1):
            result *= i
        return result
# Testing the corrected code
print(factorial(5))  # Should return 120
print(factorial(0))  # Should return 1
try:
    print(factorial(-1))  # Should raise ValueError
except ValueError as e:
    print(e)  # Output: Factorial is not defined for negative numbers.
```

OUTPUT:

```
'--' 'C:\Users\prash\OneDrive\Desktop\AIAC\Lab 10-AIAC.py'
120
1
Factorial is not defined for negative numbers.
PS C:\Users\prash\OneDrive\Desktop\AIAC>
```

Problem Statement 2: Task 2 — Improving Readability & Documentation

Scenario:The following code works but is poorly written:

.

```
def calc(a, b, c):
if c == "add":
return a + b
elif c == "sub":
return a - b
elif c == "mul":
return a * b
elif c == "div":
```

Instructions:

5. Use AI to:

o Critique the function's readability, parameter naming, and lack of documentation.

o Rewrite the function with:

1. Descriptive function and parameter names.

2. A complete docstring (description, parameters, return value, examples).

3. Exception handling for division by zero.

4. Consideration of input validation.

6. Compare the original and AI-improved versions.

7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

Expected Output:

A well-documented, robust, and readable function that handles

errors gracefully.

PROMPT:

Improve this function: Critique readability, poor naming, and lack of documentation,Rewrite using descriptive function and parameter names

Add a complete docstring (description, parameters, return, examples),Add exception handling for division by zero

#Add input validation for invalid operations and data types, Ensure the improved function is readable and robust

```
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
        return a / b
```

CODE:

ab 10-AIAC.py > calc

```python
def calculate(num1, num2, operation):
    """
    Perform a basic arithmetic operation on two numbers.

    Parameters:
    num1 (float): The first number.
    num2 (float): The second number.
    operation (str): The operation to perform. Must be one of "add", "subtract", "multiply", or "divide".

    Returns:
    float: The result of the arithmetic operation.

    Raises:
    ValueError: If the operation is not valid or if division by zero is attempted.

    Examples:
    >>> calculate(10, 5, "add")
    15
    >>> calculate(10, 5, "subtract")
    5
    >>> calculate(10, 5, "multiply")
    50
    >>> calculate(10, 5, "divide")
    2.0
    >>> calculate(10, 0, "divide")
    ValueError: Cannot divide by zero.
    >>> calculate(10, 5, "modulus")
    ValueError: Invalid operation. Supported operations are: add, subtract, multiply, divide.
    """
```

Lab 10-AIAC.py > calc

```python
def calculate(num1, num2, operation):
    if not isinstance(num1, (int, float)) or not isinstance(num2, (int, float)):
        raise ValueError("Both num1 and num2 must be numbers.")

    if operation == "add":
        return num1 + num2
    elif operation == "subtract":
        return num1 - num2
    elif operation == "multiply":
        return num1 * num2
    elif operation == "divide":
        if num2 == 0:
            raise ValueError("Cannot divide by zero.")
        return num1 / num2
    else:
        raise ValueError("Invalid operation. Supported operations are: add, subtract, multiply, divide.")
# Testing the improved function
print(calculate(10, 5, "add"))        # Should return 15
print(calculate(10, 5, "subtract"))   # Should return 5
print(calculate(10, 5, "multiply"))   # Should return 50
print(calculate(10, 5, "divide"))     # Should return 2.0
try:
    print(calculate(10, 0, "divide"))  # Should raise ValueError

except ValueError as e:
    print(e)  # Output: Cannot divide by zero.
try:
    print(calculate(10, 5, "modulus"))  # Should raise ValueError
except ValueError as e:
    print(e)  # Output: Invalid operation. Supported operations are: add, subtract, multiply, divide.
```

OUTPUT:

```
'--' 'C:\Users\prash\OneDrive\Desktop\AIAC\Lab 10-AIAC.py'
15
5
50
2.0
Cannot divide by zero.
Invalid operation. Supported operations are: add, subtract, multiply, divide.
PS C:\Users\prash\OneDrive\Desktop\AIAC>
```

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):
for i in range(2, n):
if n % i == 0:
return False
return True
```

Instructions:

8. Verify the function works correctly for sample inputs.

9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:

o List all PEP8 violations.

o Refactor the code (function name, spacing, indentation, naming).

10. Apply the AI-suggested changes and verify functionality is preserved.

11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def check_prime(n):
for i in range(2, n):
if n % i == 0:
return False
return True
```

```python
# Analyze and improve this code for PEP8 compliance

def Checkprime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

# Testing the function
print(Checkprime(11))  # Should return True
print(Checkprime(15))  # Should return False

# PEP8 Violations:
# 1. Function name should be in lowercase with underscores (check_prime)
# 2. Missing docstring
```

Lab 10-AIAC.py > ...

```python
# Refactored Code:
def check_prime(n):
    """
    Check if a number is prime.

    Parameters:
    n (int): The number to check.

    Returns:
    bool: True if prime, False otherwise.
    """
    if n <= 1:
        return False

    for i in range(2, n):
        if n % i == 0:
            return False
    return True

# Testing refactored function
print(check_prime(11))  # True
print(check_prime(15))  # False

# Note on AI in Code Reviews:
# AI can assist in code reviews by automatically identifying and suggesting fixes
# for coding standard violations such as PEP8 in Python. It improves readability,
# consistency, and reduces manual effort.
```

OUTPUT:

```
-- 'C:\Users\prash\OneDrive\Desktop\AIAC\Lab 10-AIAC.py'
True
False
True
False
PS C:\Users\prash\OneDrive\Desktop\AIAC>
```

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):
return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:

o Readability and naming.

o Reusability and modularity.

o Edge cases (non-list input, empty list, non-integer elements).

2. Use AI to generate a code review covering:

a. Better naming and function purpose clarity.

b. Input validation and type hints.

c. Suggestions for generalization (e.g., configurable multiplier).

3. Refactor the function based on AI feedback.

4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

An improved function with type hints, validation, and clearer intent, e.g.:

```
from typing import List, Union
def double_even_numbers(numbers: List[Union[int,
float]]) -> List[Union[int, float]]:
if not isinstance(numbers, list):
```

raise TypeError("Input must be a list")

return [num * 2 for num in numbers if isinstance(num,

(int, float)) and num % 2 == 0]

```python
#Review and improve this function:Analyze readability and naming,Check reusability and modularity,Handle edge cases (
#Add type hints and input validation,Improve function name and clarity,Allow configurable multiplier
#Refactor the function based on improvements,Add a short reflection on AI as reviewer vs assistant

def processData(d):
    return [x * 2 for x in d if x % 2 == 0]
# Critique: The function name "processData" is vague and does not clearly indicate what the function does. The parame
# Improved Code:
from typing import List
def double_even_numbers(numbers: List[int], multiplier: int = 2) -> List[int]:
    """
    Double the even numbers in a list.

    Parameters:
    numbers (List[int]): A list of integers to process.
    multiplier (int): The factor by which to multiply the even numbers. Default is 2.

    Returns:
    List[int]: A list of doubled even numbers.

    Raises:
    ValueError: If the input is not a list or contains non-integer elements.
    """
    if not isinstance(numbers, list):
        raise ValueError("Input must be a list.")

    if not all(isinstance(x, int) for x in numbers):
        raise ValueError("All elements in the list must be integers.")

    return [x * multiplier for x in numbers if x % 2 == 0]
# Testing the improved function
```

```python
def double_even_numbers(numbers: List[int], multiplier: int = 2) -> List[int]:

    if not all(isinstance(x, int) for x in numbers):
        raise ValueError("All elements in the list must be integers.")

    return [x * multiplier for x in numbers if x % 2 == 0]
# Testing the improved function
print(double_even_numbers([1, 2, 3, 4, 5]))  # Should return [4, 8]
print(double_even_numbers([1, 2, 3, 4, 5], multiplier=3))  # Should return [6, 12]
try:
    print(double_even_numbers("not a list"))  # Should raise ValueError
except ValueError as e:
    print(e)  # Output: Input must be a list.
try:
    print(double_even_numbers([1, 2, "three", 4]))  # Should raise ValueError
except ValueError as e:
    print(e)  # Output: All elements in the list must be integers.
```

OUTPUT:

```
C:\Users\prash\OneDrive\Desktop\AIAC\Lab 10-AIAC.py
[4, 8]
[6, 12]
Input must be a list.
All elements in the list must be integers.
PS C:\Users\prash\OneDrive\Desktop\AIAC>
```

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
total = 0
for num in numbers:
total += num ** 2
return total
```

Instructions:

1. Test the function with a large list (e.g., range(1000000)).
2. Use AI to:
   - o Analyze time complexity.
   - o Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
   - o Provide an optimized version.
3. Compare execution time before and after optimization.
4. Discuss trade-offs between readability and performance.

Expected Output:

An optimized function, such as:

```
def sum_of_squares_optimized(numbers):
return sum(x * x for x in numbers)
```

```python
#Analyze and optimize this function:Test with a large dataset (e.g., range(1000000)),Analyze time complexity
#Suggest performance improvements,Provide an optimized version,Compare execution time before and after optimization
#Discuss readability vs performance trade-offs

def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
    return total
# Testing the function with a large dataset
import time
start_time = time.time()
print(sum_of_squares(range(1000000)))  # Should return the sum of squares
end_time = time.time()
print(f"Execution time before optimization: {end_time - start_time} seconds")
# Time Complexity Analysis: The time complexity of the original function is O(n) because it iterates through each number in th
# Performance Improvements: We can optimize the function by using a generator expression instead of a list comprehension
def optimized_sum_of_squares(numbers):
    return sum(num ** 2 for num in numbers)
# Testing the optimized function
start_time = time.time()
print(optimized_sum_of_squares(range(1000000)))  # Should return the same
end_time = time.time()
print(f"Execution time after optimization: {end_time - start_time} seconds")
```

EMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

OUTPUT:

```
'--' 'C:\Users\prash\OneDrive\Desktop\AIAC\Lab 10-AIAC.py'
333332833333500000
Execution time before optimization: 0.05612921714782715 seconds
333332833333500000
Execution time after optimization: 0.06907534599304199 seconds
PS C:\Users\prash\OneDrive\Desktop\AIAC>
```